

MATH 8090: Stationary processes and Model-Free Estimation of Stationary Means and Covariances

Whitney Huang, Clemson University

9/9-9/11/2026

Contents

Stationary Processes	1
Examples of MA(q) processes	1
Examples of AR(2) processes	3
Checking AR(2) Stationarity via the Characteristic Equation	5
Model-Free Estimation of Stationary Means and Covariances	8
Mean Estimation	8
Autocovariance Function (ACVF)	10
Autocorrelation Function (ACF)	12
Lake Huron Example	13
Box test for temporal independence	16
Box and Pierce test Box and Pierce (1970)	16
Ljung-Box Test	16
References	19

Stationary Processes

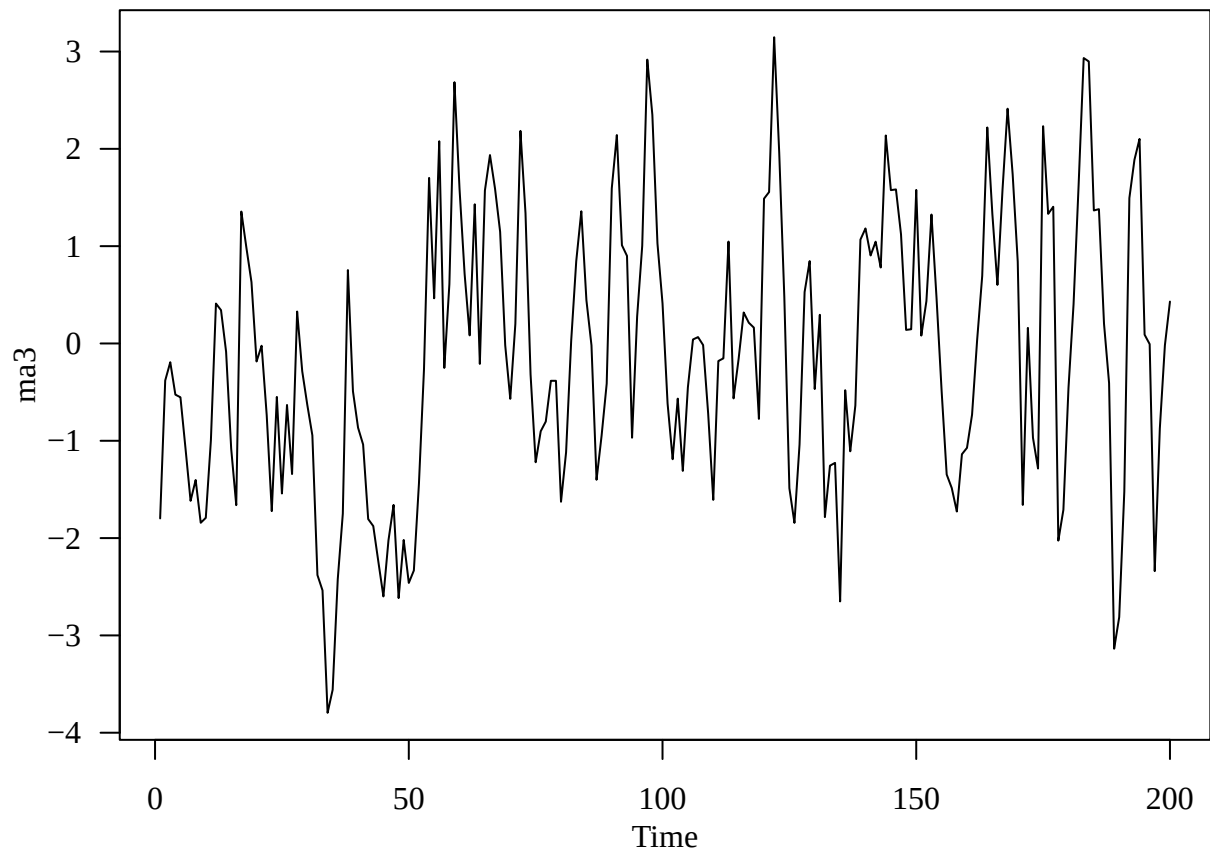
Examples of MA(q) processes

```
# Set the random seed for reproducibility
set.seed(1234)

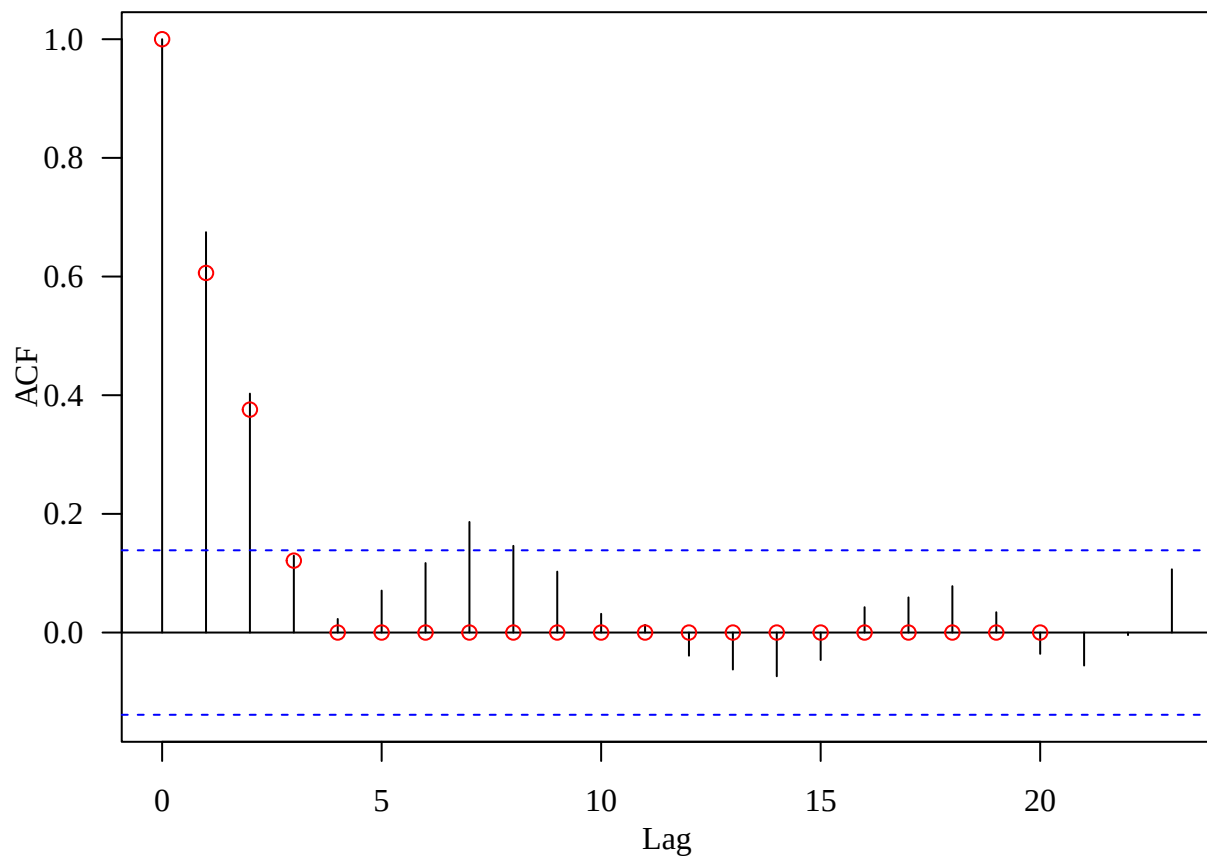
# Simulate 200 observations from an MA(3) process
#  $X_t = Z_t + 0.6 Z_{t-1} + 0.5 Z_{t-2} + 0.2 Z_{t-3}$ 
ma3 <- arima.sim(n = 200, list(ma = c(0.6, 0.5, 0.2)))

# Set graphical parameters
par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif")

# Plot the simulated MA(3) time series
ts.plot(ma3)
```



```
# Plot the sample ACF  
acf(ma3)  
  
# Compute the theoretical ACF of the MA(3) process  
acf_true <- ARMAacf(ma = c(0.6, 0.5, 0.2), lag.max = 20)  
  
# Add the theoretical ACF (red points) to the sample ACF plot  
points(0:20, acf_true, col = "red")
```

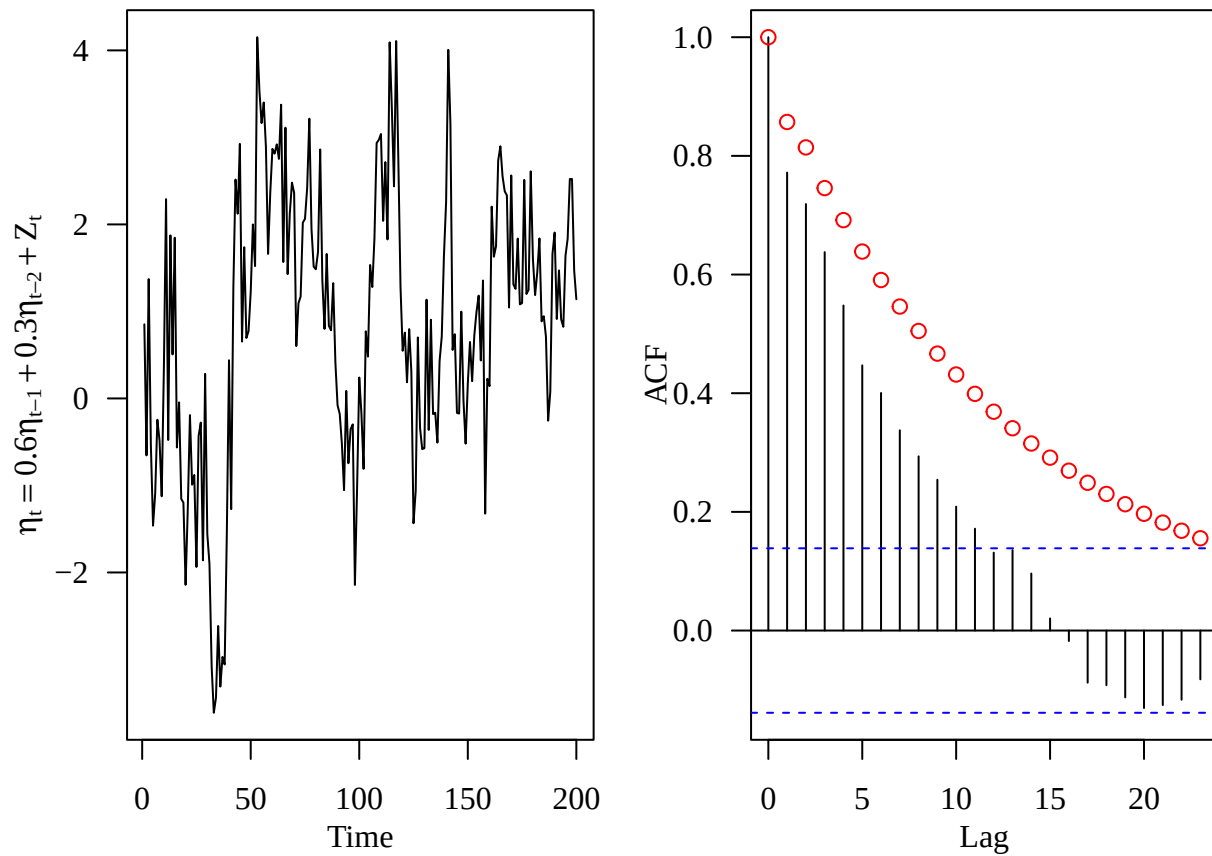


Examples of AR(2) processes

```
par(mfrow = c(1, 2), mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1,
    family = "serif")

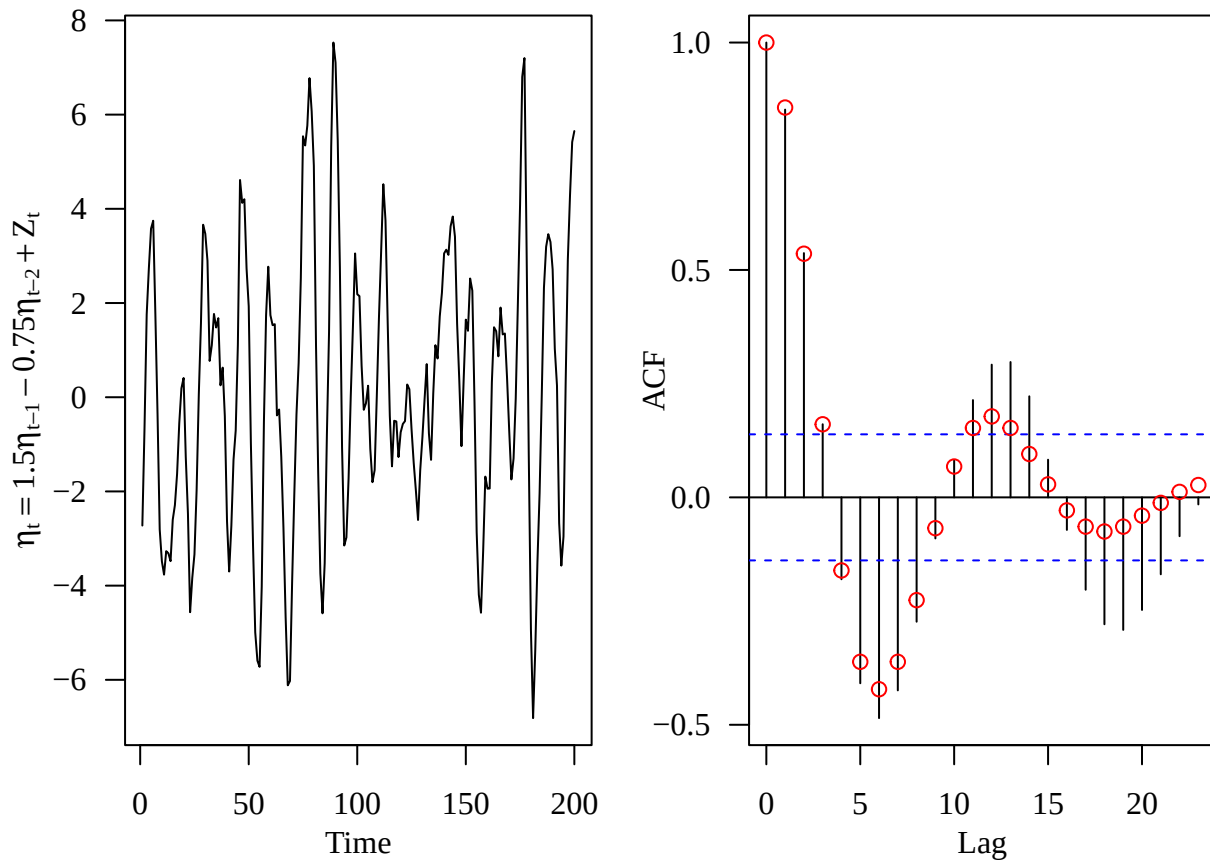
# Simulate 200 observations from an AR(2) process
#  $X_t = 0.6 X_{t-1} + 0.3 X_{t-2} + Z_t$ 
set.seed(2026)
ar2_1 <- arima.sim(n = 200, list(ar = c(0.6, 0.3)))

ts.plot(ar2_1, ylab = expression(eta[t] == 0.6*eta[t-1] + 0.3*eta[t-2] + Z[t]))
acf(ar2_1)
points(0:23, ARMAacf(ar = c(0.6, 0.3), lag.max = 23), col = "red")
```



```
# Simulate 200 observations from another AR(2) process
#  $X_t = 1.5 X_{t-1} - 0.75 X_{t-2} + Z_t$ 
set.seed(2026)
ar2_2 <- arima.sim(n = 200, list(ar = c(1.5, -0.75)))

ts.plot(ar2_2, ylab = expression(eta[t] == 1.5*eta[t-1] - 0.75*eta[t-2] + Z[t]))
acf(ar2_2)
points(0:23, ARMAacf(ar = c(1.5, -0.75), lag.max = 23), col = "red")
```



Checking AR(2) Stationarity via the Characteristic Equation

Here we are going to visualize AR(2) roots of $1 - \phi_1 z - \phi_2 z^2 = 0$

Stationary $\iff$ all roots have modulus > 1

Below is an R function:

```
plot_ar_stationarity <- function(phi, main = NULL) {
  stopifnot(is.numeric(phi), length(phi) >= 1)
  p <- length(phi)

  # AR polynomial coefficients in z: 1 - phi_1 z - ... - phi_p z^p
  coefs <- c(1, -phi)
  r <- polyroot(coefs)          # complex roots
  mod <- Mod(r)
  bad <- mod <= 1 + 1e-10      # on/inside unit circle => nonstationary

  # Plot setup
  R <- max(1, max(mod))
  lim <- 1.2 * R
  if (is.null(main)) {
    main <- sprintf("AR(%d) roots of 1 - sum phi_i z^i", p)
  }
  plot(NA, xlim = c(-lim, lim), ylim = c(-lim, lim),
       xlab = "Re(z)", ylab = "Im(z)", asp = 1, main = main)
```

```

abline(h = 0, v = 0, col = "gray80")
th <- seq(0, 2 * pi, length.out = 512)
lines(cos(th), sin(th), lty = 2) # unit circle

# Roots
cols <- ifelse(bad, "red", "lightblue")
points(Re(r), Im(r), pch = 21, bg = cols, cex = 1.3)
text(Re(r), Im(r), labels = sprintf("|r|=%.2f", mod), pos = 3, cex = 0.8)

legend("topleft", bty = "n",
      legend = c("Unit circle |z|=1", "Root |z| <= 1 (nonstationary)", "Root |z| > 1"),
      lty = c(2, NA, NA), pch = c(NA, 21, 21), pt.bg = c(NA, "red", "lightblue"))

msg <- if (any(bad)) "Not stationary: a root lies on/inside |z|=1."
      else           "Stationary: all roots outside |z|=1."
mtext(msg, line = 0.5, col = if (any(bad)) "red" else "darkgreen")

invisible(list(roots = r, modulus = mod, stationary = !any(bad)))
}

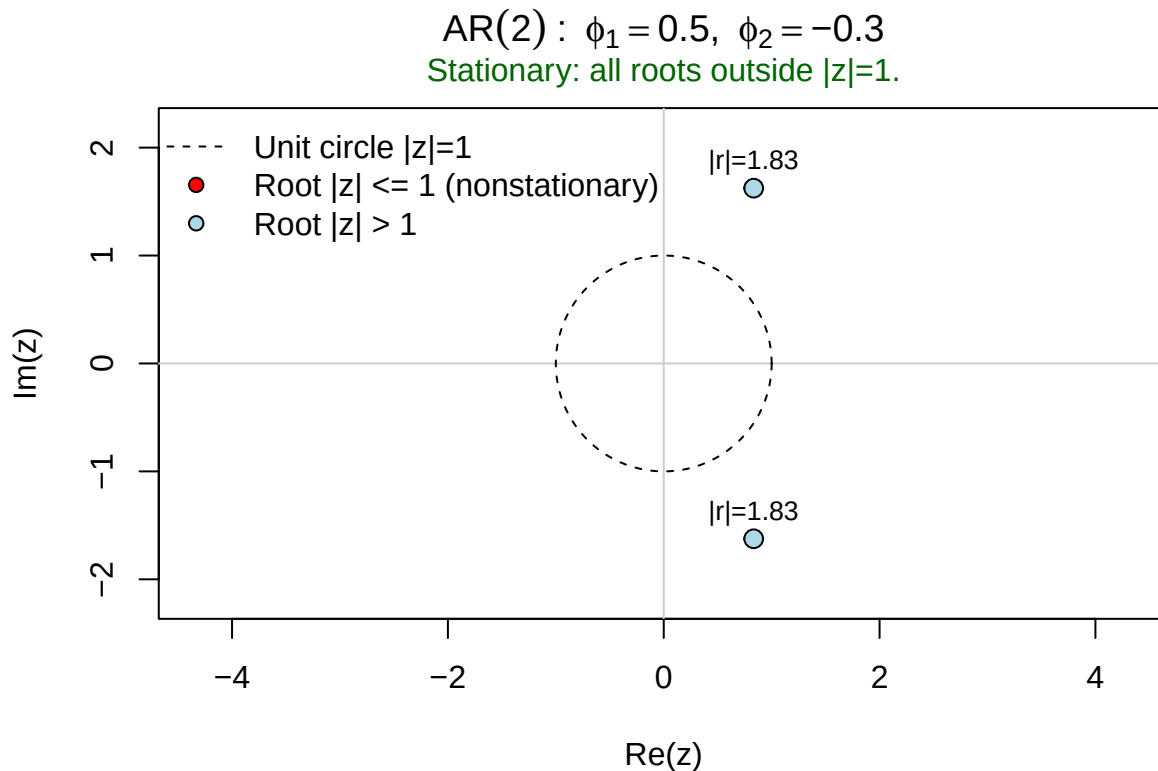
```

Try a few examples to see whether this works:

```

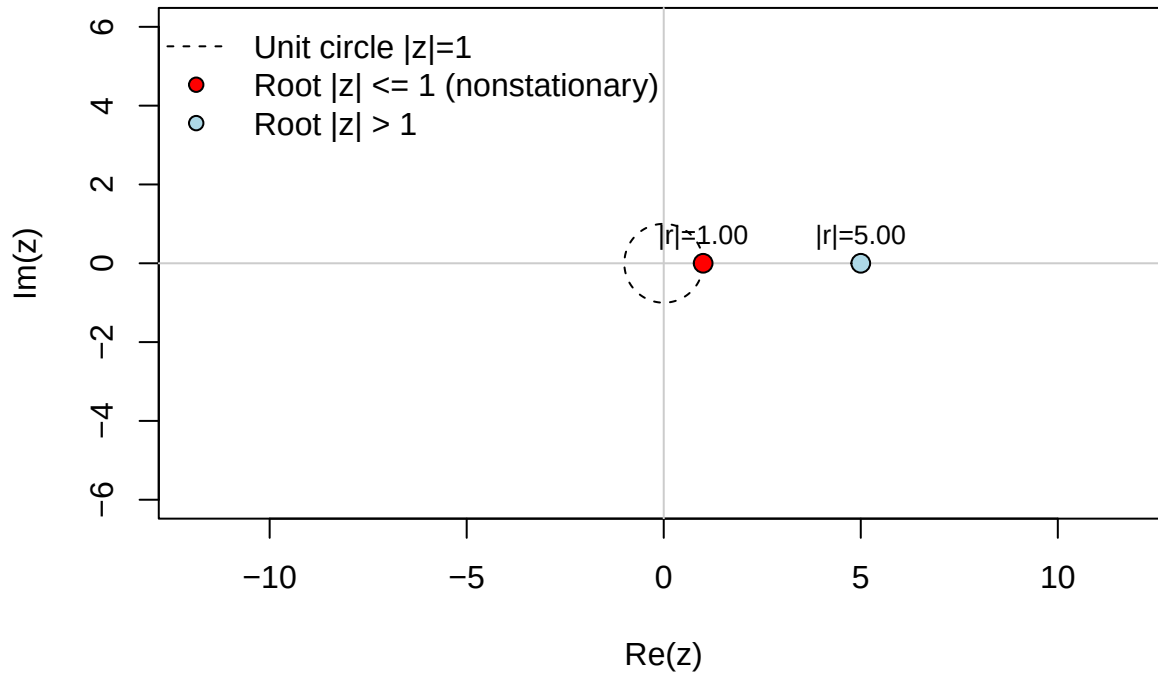
# Example 1: Check stationarity of an AR(2) process
# with phi_1 = 0.5 and phi_2 = -0.3
plot_ar_stationarity(
  c(0.5, -0.3),
  main = bquote(AR(2) ~ ": " ~ phi[1] == .(0.5) * ", " ~ phi[2] == .(-0.3))
)

```



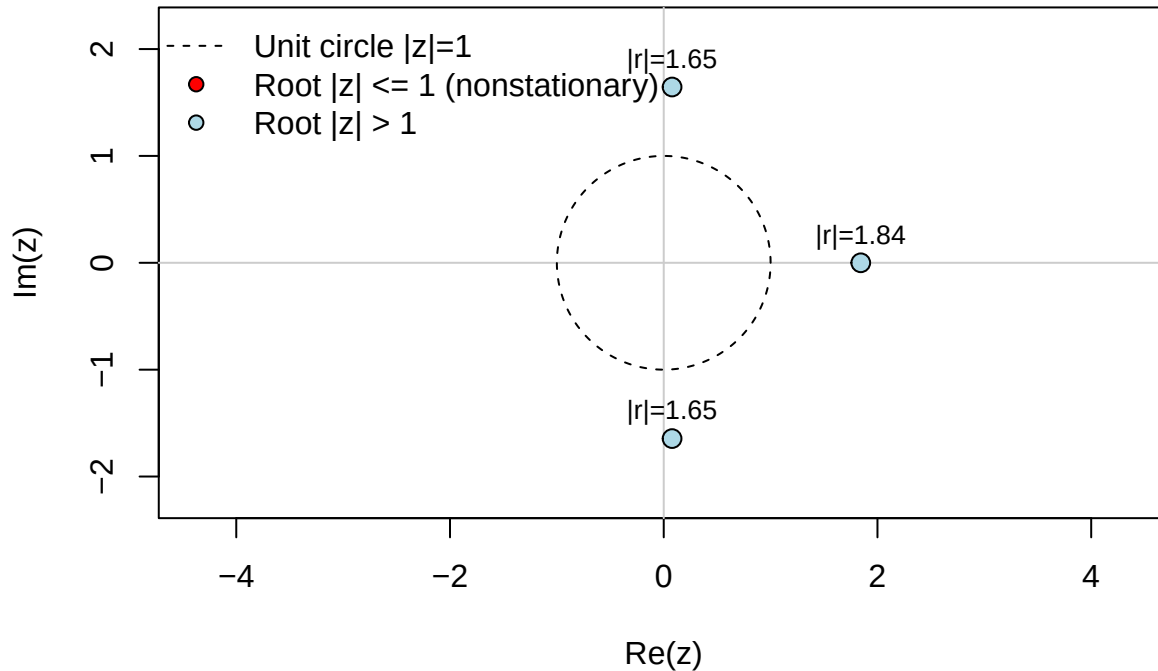
```
# Example 2: Check stationarity of an AR(2) process
# with phi_1 = 1.2 and phi_2 = -0.2
plot_ar_stationarity(
  c(1.2, -0.2),
  main = bquote(AR(2) ~ ": " ~ phi[1] == .(1.2) * ", " ~ phi[2] == .(-0.2))
)
```

AR(2) : $\phi_1 = 1.2$, $\phi_2 = -0.2$
 Not stationary: a root lies on/inside $|z|=1$.



```
# Example 3: Check stationarity of an AR(3) process
# with phi_1 = 0.6, phi_2 = -0.4, and phi_3 = 0.2
plot_ar_stationarity(
  c(0.6, -0.4, 0.2),
  main = bquote(AR(3) ~ ": " ~
    phi[1] == .(0.6) * ", " ~
    phi[2] == .(-0.4) * ", " ~
    phi[3] == .(0.2))
)
```

AR(3) : $\phi_1 = 0.6$, $\phi_2 = -0.4$, $\phi_3 = 0.2$
 Stationary: all roots outside $|z|=1$.



Model-Free Estimation of Stationary Means and Covariances

Mean Estimation

Given a stationary process $\{\eta_t\}_{t=1}^T$, the point estimator is $\bar{\eta} = \frac{1}{T} \sum_{t=1}^T \eta_t$. The variance of this estimator is

$$\nu_T = \text{Var}(\bar{\eta}) = \text{Var}\left(\frac{1}{T} \sum_{t=1}^T \eta_t\right) = \frac{1}{T} \sum_{h=-(T-1)}^{T-1} \left(1 - \frac{|h|}{T}\right) \gamma(h)$$

```
# Monte Carlo approximation of the sampling distribution of the sample mean
M = 1000; T = 200; phi = 0.9

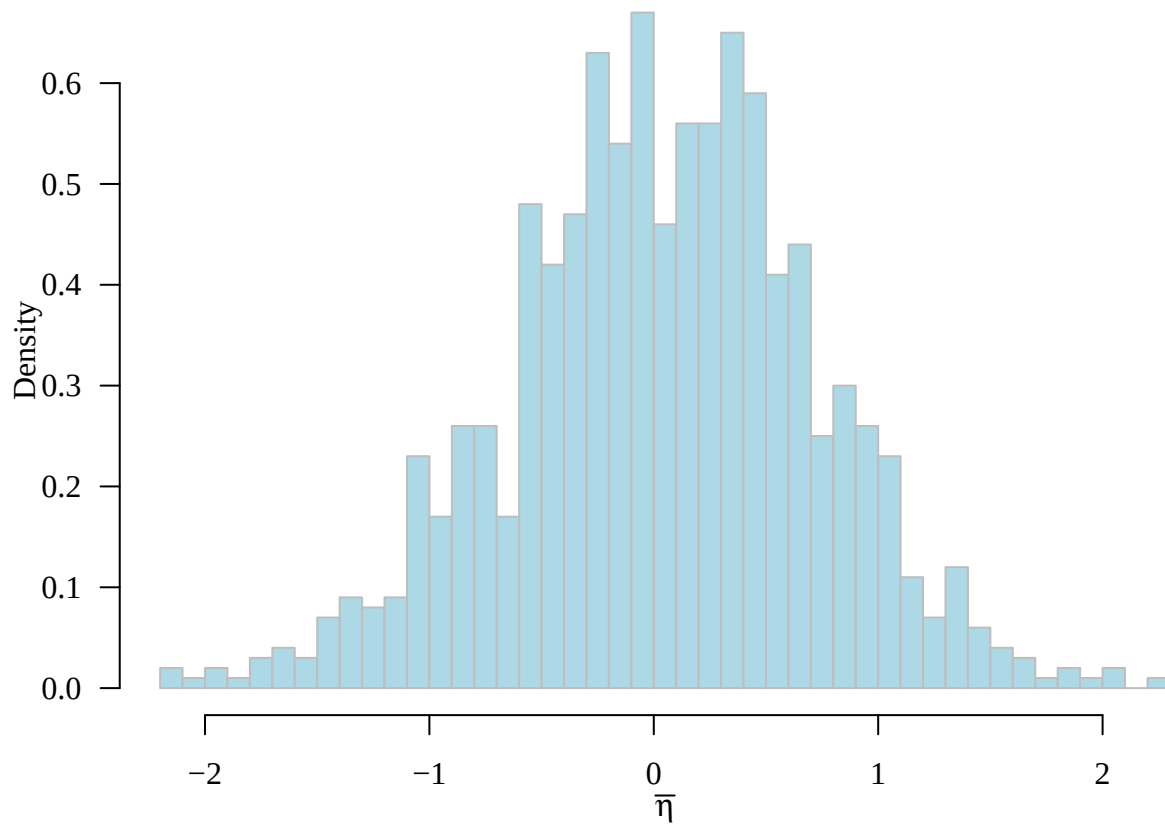
# Set the random seed for reproducibility
set.seed(123)

# Simulate M independent AR(1) time series, each of length T
sim <- replicate(M, arima.sim(n = T, list(ar = c(phi))))

# Compute the sample mean for each simulated time series
eta_bar <- apply(sim, 2, mean)

# Set graphical parameters
par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif")

# Plot the Monte Carlo distribution of the sample mean
hist(eta_bar, 40, col = "lightblue", border = "gray", las = 1, prob = T,
     main = "", xlab = expression(bar(eta)))
```



```
# Estimate the variance of the sample mean using the Monte Carlo samples
(nu_T_hat <- var(eta_bar))
```

```
## [1] 0.4659671
```

```
par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0), las = 1, family = "serif")
```

```
# Plot the Monte Carlo distribution of the sample mean
```

```
hist(eta_bar, 40, col = "lightblue", border = "gray", las = 1, prob = T,
     main = "", xlab = expression(bar(eta)))
```

```
# Estimate the density of the Monte Carlo sample means
```

```
kde <- density(eta_bar)
```

```
# Superimpose the estimated density curve
```

```
lines(kde, col = "blue")
```

```
# Compute the theoretical sampling distribution
```

```
mu = 0
```

```
h <- -(T-1):(T-1)
```

```
# Compute the exact finite-sample variance of the sample mean
```

```
nu_T <- (1 / T) * sum((1 - (abs(h) / T)) * (phi^(abs(h)) / (1 - phi^2)))
```

```
# Create a grid for plotting the theoretical density
```

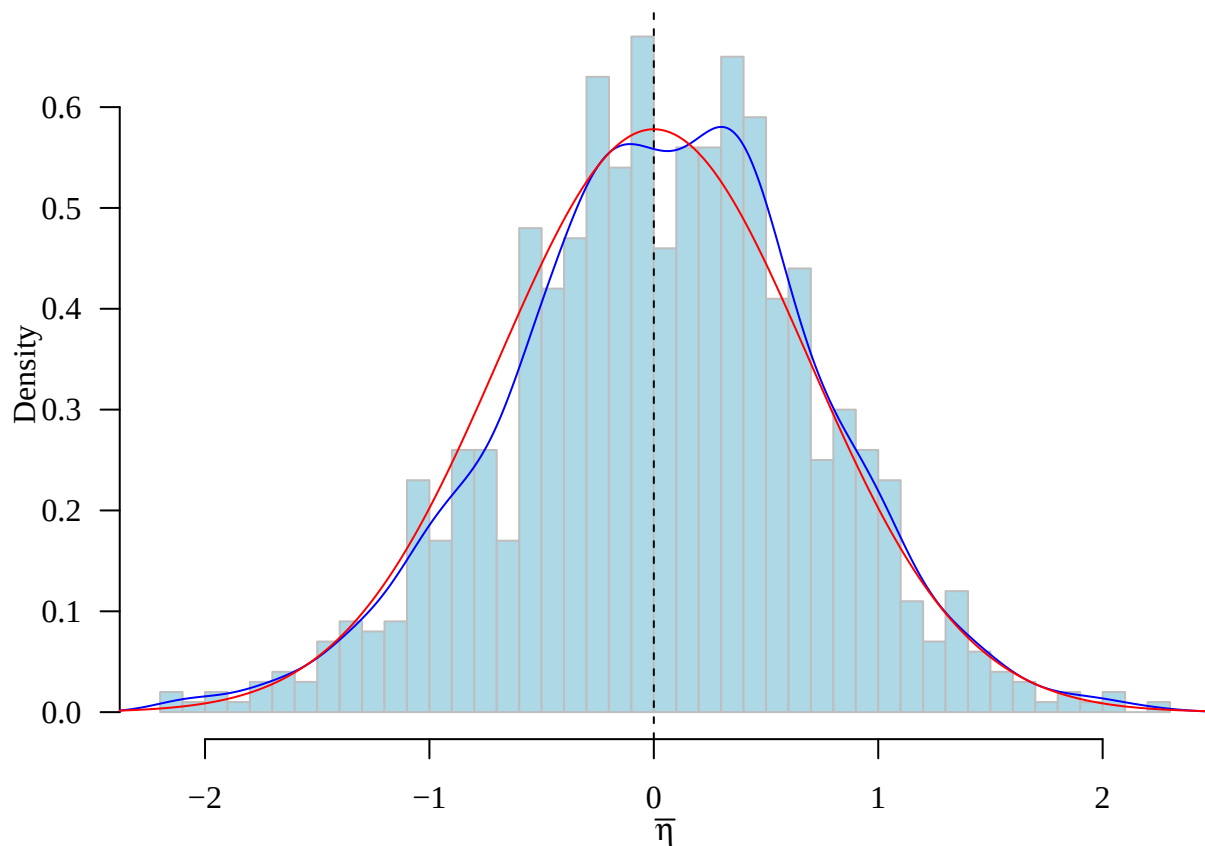
```
xg <- seq(-2.5, 2.5, 0.01)
```

```

# Mark the theoretical mean
abline(v = mu, lty = 2)

# Superimpose the theoretical normal density
lines(xg, dnorm(xg, sd = sqrt(nu_T)), col = "red")

```



```
## Compare the finite-sample and limiting variance approximations
```

```

# Compute the exact finite-sample variance of the sample mean
(nu_T <- (1 / T) * sum((1 - (abs(h) / T)) *
  (phi^(abs(h)) / (1 - phi^2))))

```

```
## [1] 0.4763158
```

```

# Compute the large-sample approximation to the variance of the sample mean
(nu <- (1 / T) * (1 / (1 - phi)^2))

```

```
## [1] 0.5
```

Autocovariance Function (ACVF)

```

# Estimate the ACVF for each simulated time series
gamma <- apply(sim, 2, function(x)

```

```

acf(x, plot = FALSE, type = "covariance")$acf)

# Compute the average sample ACVF across Monte Carlo replications
(est <- apply(gamma, 1, mean))

## [1] 4.762396047 4.210822098 3.718189717 3.276955489 2.883257698 2.529033768
## [7] 2.215391725 1.934657288 1.684410252 1.460895692 1.261443145 1.085257775
## [13] 0.927192703 0.791189571 0.671906177 0.563884323 0.468728429 0.380609541
## [19] 0.300366800 0.227258194 0.161799558 0.106047046 0.056009807 0.007812902

# Compute the theoretical ACVF of the AR(1) process
(true <- ARMAacf(ar = c(0.9), lag.max = 23) * (1 / (1 - phi^2)))

##      0      1      2      3      4      5      6      7
## 5.2631579 4.7368421 4.2631579 3.8368421 3.4531579 3.1078421 2.7970579 2.5173521
##      8      9     10     11     12     13     14     15
## 2.2656169 2.0390552 1.8351497 1.6516347 1.4864712 1.3378241 1.2040417 1.0836375
##     16     17     18     19     20     21     22     23
## 0.9752738 0.8777464 0.7899718 0.7109746 0.6398771 0.5758894 0.5183005 0.4664704

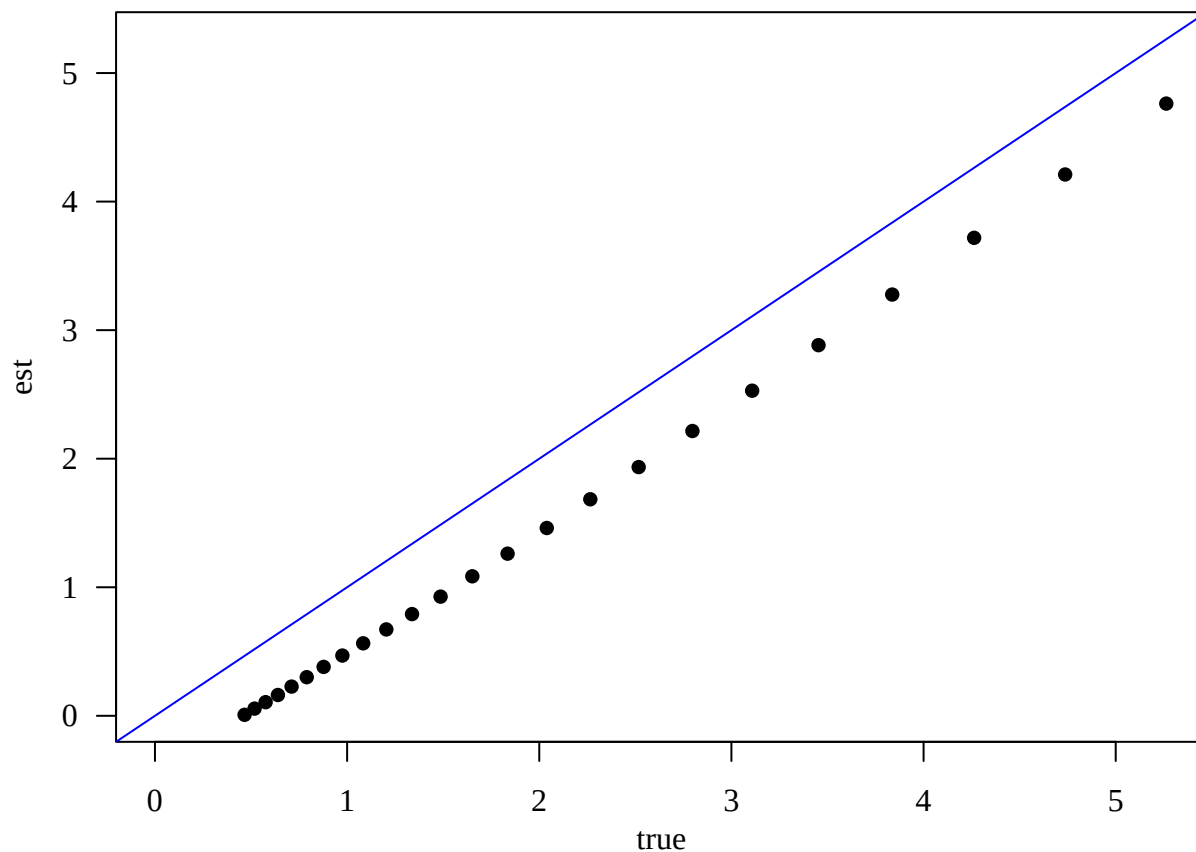
# Compare the estimated and theoretical ACVFs
# The sample ACVF is a biased estimator
rg <- range(true, est)

par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0),
    las = 1, family = "serif")

# Plot the average sample ACVF against the theoretical ACVF
plot(true, est, pch = 16, xlim = rg, ylim = rg, las = 1)

# Add the 45-degree reference line (perfect agreement)
abline(0, 1, col = "blue")

```



```
# Verify that the theoretical ACVF is non-negative definite
```

```
# Construct the matrix of pairwise time lags
```

```
h <- outer(1:24, 1:24, "-")
```

```
# Construct the theoretical covariance matrix for the AR(1) process
```

```
Sigma <- phi^abs(h) / (1 - phi^2)
```

```
# Check that all eigenvalues of the covariance matrix are non-negative
```

```
eigen(Sigma)$values
```

```
## [1] 64.7315984 26.5011371 11.7018213 6.1924112 3.7670568 2.5262589
## [7] 1.8165664 1.3759195 1.0849122 0.8833909 0.7385536 0.6313427
## [13] 0.5500994 0.4873821 0.4382698 0.3994115 0.3684691 0.3437787
## [19] 0.3241390 0.3086742 0.2967447 0.2878873 0.2817747 0.2781898
```

Autocorrelation Function (ACF)

Population ACF

$$\rho(h) = \text{Cor}(\eta_t, \eta_{t+h}) = \frac{\mathbb{E}[(\eta_t - \mu)(\eta_{t+h} - \mu)]}{\sqrt{\text{Var}(\eta_t)\text{Var}(\eta_{t+h})}}$$

Sample ACF

$$\hat{\rho}(h) = \frac{\hat{\gamma}(h)}{\hat{\gamma}(0)},$$

where $\hat{\gamma}(h) = \frac{1}{T} \sum_{t=1}^{T-|h|} (\eta_t - \bar{\eta})(\eta_{t+h} - \bar{\eta})$.

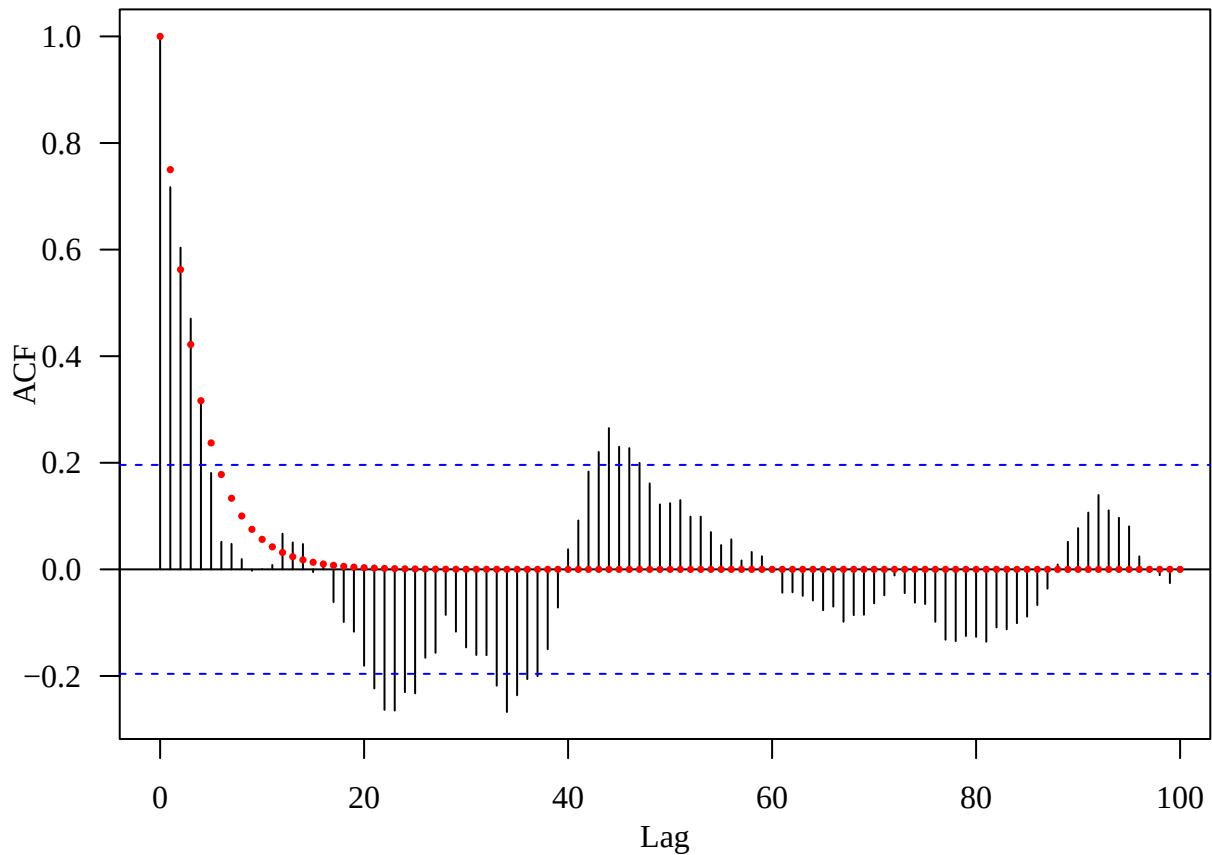
```
# Simulate an AR(1) process with phi = 0.75
set.seed(2026)
AR1 <- arima.sim(n = 100, list(ar = c(0.75)))

par(mar = c(3, 3.5, 0.5, 0.6), mgp = c(2, 1, 0),
    las = 1, family = "serif")

# Plot the sample ACF of the simulated AR(1) process
acf(AR1, lag = 100)

# Compute the theoretical ACF of the AR(1) process
acf_true <- ARMAacf(ar = c(0.75), lag.max = 100)

# Superimpose the theoretical ACF on the sample ACF
points(0:100, acf_true, pch = 16, cex = 0.5, col = "red")
```



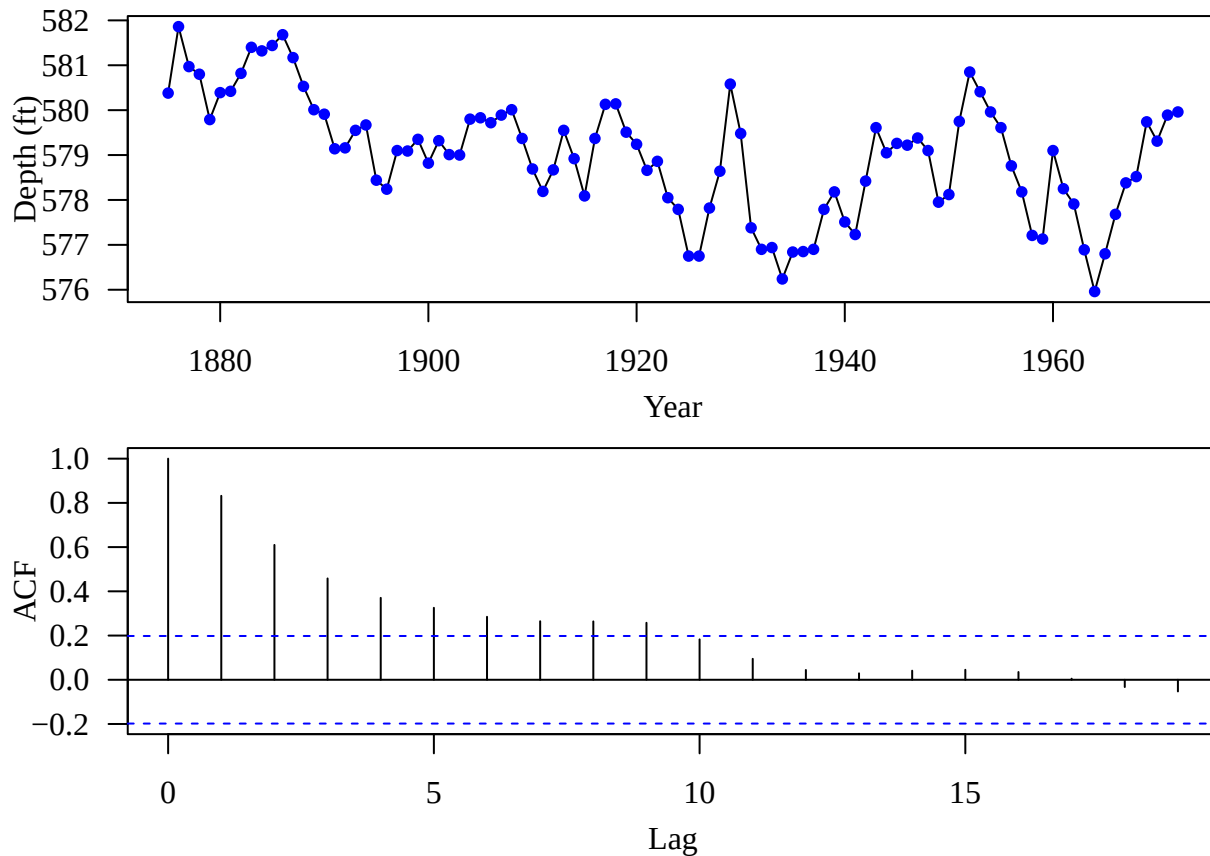
Lake Huron Example

```

# Load the Lake Huron annual water-level data
data(LakeHuron)

# Plot the original time series and its sample ACF
par(mar = c(3.3, 3.5, 0.5, 0.6), mgp = c(2.2, 1, 0), mfrow = c(2, 1),
    las = 1, family = "serif")
plot(LakeHuron, ylab = "Depth (ft)", xlab = "Year")
points(LakeHuron, cex = 0.8, col = "blue", pch = 16)
acf(LakeHuron)

```

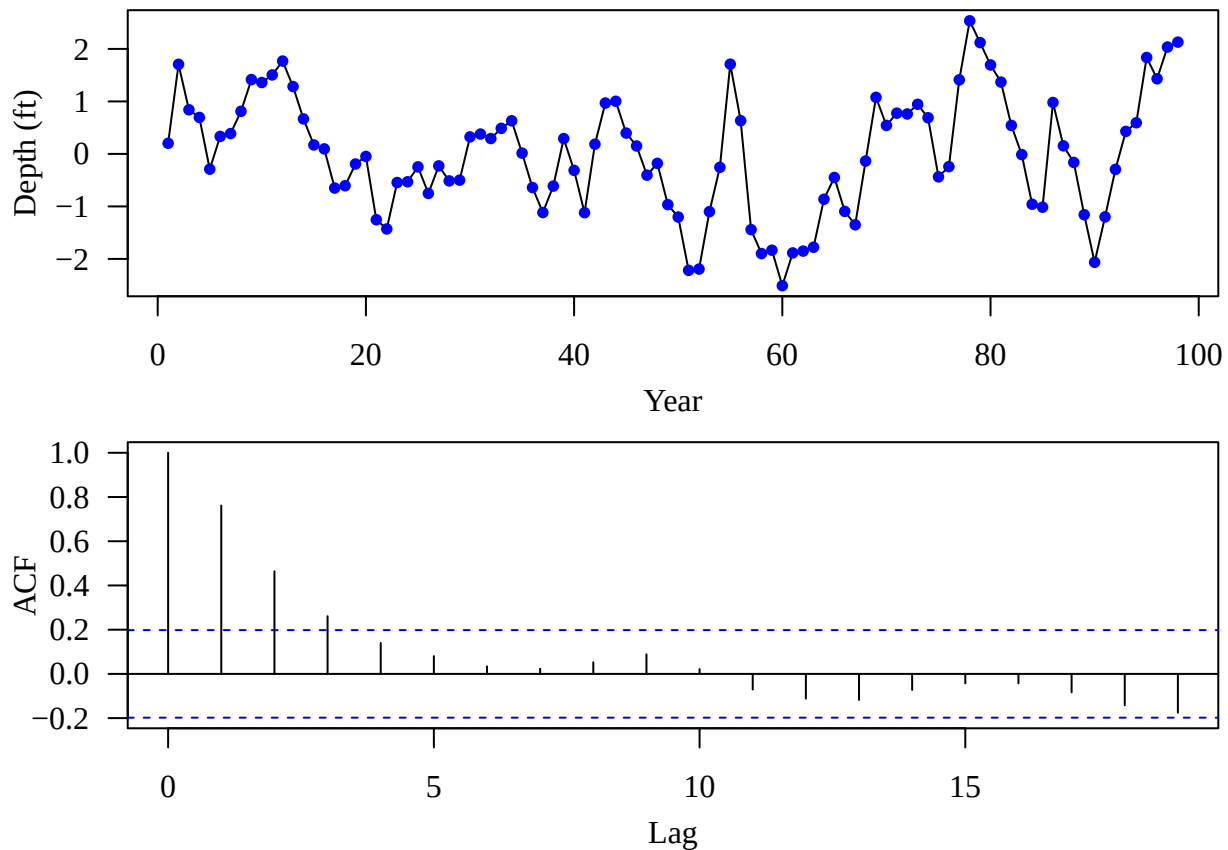


```

# Remove the linear trend using least squares regression
yr <- 1875:1972
lm <- lm(LakeHuron ~ yr)

# Examine the detrended series (residuals) and its sample ACF
par(mar = c(3.3, 3.5, 0.5, 0.6), mgp = c(2.2, 1, 0), mfrow = c(2, 1),
    las = 1, family = "serif")
plot(lm$residuals, ylab = "Depth (ft)", xlab = "Year", type = "l")
points(lm$residuals, cex = 0.8, col = "blue", pch = 16)
acf(lm$residuals)

```

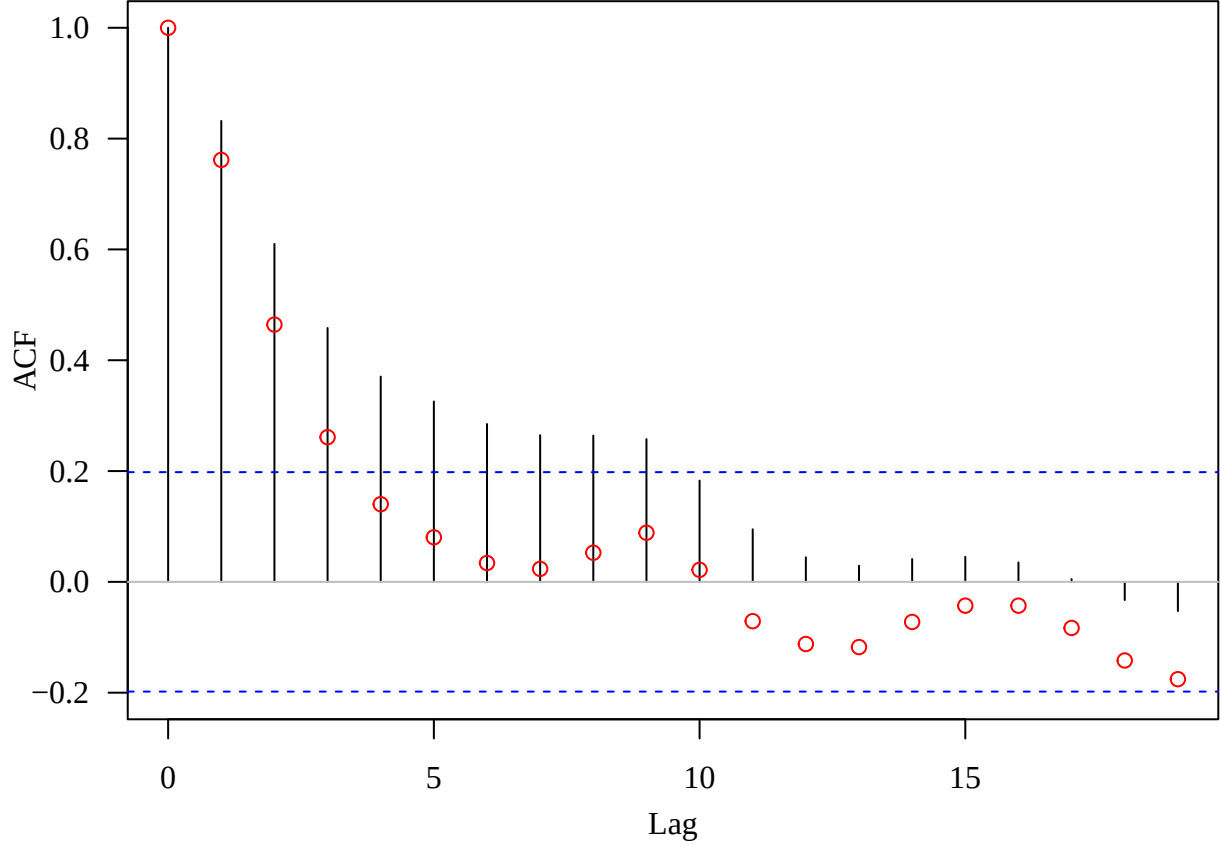


```
# Compare the sample ACFs before and after detrending
par(mar = c(3.3, 3.5, 0.5, 0.6), mgp = c(2.2, 1, 0), mfrow = c(1, 1),
    las = 1, family = "serif")

# ACF of the original series
plot(0:19, acf(LakeHuron, plot = FALSE)$acf, type = "h",
     xlab = "Lag", ylab = "ACF", ylim = c(-0.2, 1))

# Add zero reference line and approximate 95% bounds
abline(h = 0, col = "gray")
abline(h = c(-1, 1) * qnorm(0.975) / sqrt(length(LakeHuron)),
      col = "blue", lty = 2)

# Add the ACF of the detrended series for comparison
acf_detrend <- acf(lm$residuals, plot = FALSE)$acf
points(0:19, acf_detrend, col = "red")
```



Box test for temporal independence

Box and Pierce test Box and Pierce (1970)

We wish to test:

$H_0 : \{\eta_1, \eta_2, \dots, \eta_T\}$ is an i.i.d. noise sequence

$H_1 : H_0$ is false

1. Under H_0 ,

$$\hat{\rho}(h) \sim N(0, \frac{1}{T}) \stackrel{d}{=} \frac{1}{\sqrt{n}} N(0, 1)$$

2. Hence

$$Q = T \sum_{i=1}^k \hat{\rho}^2(h) \sim \chi_{df=k}^2$$

3. We reject H_0 if $Q > \chi_k^2(1 - \alpha)$, the $1 - \alpha$ quantile of the chi-squared distribution with k degrees of freedom

Ljung-Box Test

$$Q_{LB} = T(T-2) \sum_{h=1}^k \frac{\hat{\rho}^2(h)}{n-h} \sim \chi_k^2.$$

The Ljung-Box test Ljung and Box (1978) can be more powerful than the Box and Pierce test

```

# Generate a white noise series
set.seed(123)
test <- rnorm(100)

# Box-Pierce portmanteau test
Box.test(test, lag = 10, type = "Box-Pierce")

##
## Box-Pierce test
##
## data: test
## X-squared = 5.2889, df = 10, p-value = 0.8711

# Ljung-Box test
Box.test(test, lag = 10, type = "Ljung-Box")

##
## Box-Ljung test
##
## data: test
## X-squared = 5.6107, df = 10, p-value = 0.8468

# Monte Carlo experiment:
# Compare the p-value distributions of the two tests under H0
M <- 10000

p.bp <- numeric(M)
p.lb <- numeric(M)

for (i in 1:M) {
  # Simulate a white noise series under H0
  x <- rnorm(100)
  # Store p-values from the two portmanteau tests
  p.bp[i] <- Box.test(x, lag = 5, type = "Box-Pierce")$p.value
  p.lb[i] <- Box.test(x, lag = 5, type = "Ljung-Box")$p.value
}

# Compare the empirical p-value distributions
par(mar = c(3.5, 3.5, 0.5, 0.6), mgp = c(2, 0.7, 0), mfrow = c(1, 2),
    las = 1, family = "serif")

hist(p.bp, breaks = 40, probability = TRUE, col = "lightblue", border = "gray",
     xlim = c(0, 1), ylim = c(0, 1.5), xlab = "p-value", main = "")
abline(h = 1, col = "red", lty = 2, lwd = 2)

legend("topright", legend = c("Box-Pierce", "Uniform(0,1)"),
     fill = c("lightblue", NA), border = c("gray", NA),
     lty = c(NA, 2), col = c(NA, "red"), lwd = c(NA, 2), bty = "n")

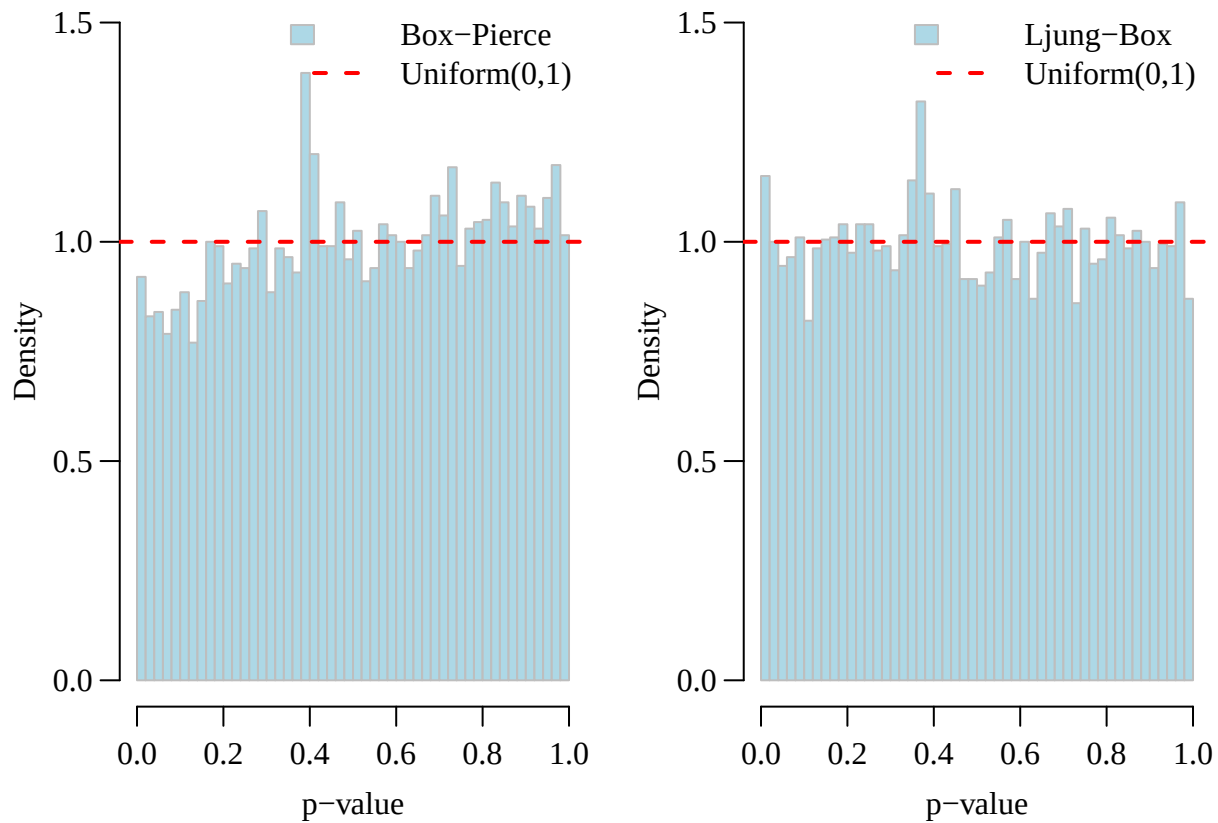
hist(p.lb, breaks = 40, probability = TRUE, col = "lightblue", border = "gray",
     xlim = c(0, 1), ylim = c(0, 1.5), xlab = "p-value", main = "")
abline(h = 1, col = "red", lty = 2, lwd = 2)

```

```

legend("topright", legend = c("Ljung-Box", "Uniform(0,1)"),
      fill = c("lightblue", NA), border = c("gray", NA),
      lty = c(NA, 2), col = c(NA, "red"), lwd = c(NA, 2), bty = "n")

```



```

# Numerical summaries

```

```

summary(p.bp)

```

```

##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## 0.0000156 0.2795561 0.5204488 0.5188547 0.7681540 0.9999675

```

```

summary(p.lb)

```

```

##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## 0.0000076 0.2513666 0.4913277 0.4974939 0.7475942 0.9999630

```

```

# Empirical rejection rates at alpha = 0.05

```

```

mean(p.bp < 0.05)

```

```

## [1] 0.0424

```

```

mean(p.lb < 0.05)

```

```

## [1] 0.0532

```

```
# Box-Pierce test for residual serial correlation up to lag 10
Box.test(lm$residuals, lag = 10, type = "Box-Pierce")
```

```
##
## Box-Pierce test
##
## data:  lm$residuals
## X-squared = 88.469, df = 10, p-value = 1.077e-14
```

```
# Ljung-Box test with a finite-sample correction
Box.test(lm$residuals, lag = 10, type = "Ljung-Box")
```

```
##
## Box-Ljung test
##
## data:  lm$residuals
## X-squared = 91.776, df = 10, p-value = 2.331e-15
```

```
# Weighted portmanteau test of Fisher and Gallagher (2012)
library(WeightedPortTest)
Weighted.Box.test(lm$residuals, lag = 10)
```

```
##
## Weighted Box-Pierce test (Gamma Approximation)
##
## data:  lm$residuals
## Weighted X-squared on Residuals for fitted ARMA process = 83.252, Shape
## = 3.9286, Scale = 1.4000, p-value < 2.2e-16
```

References

- Box, George EP, and David A Pierce. 1970. "Distribution of Residual Autocorrelations in Autoregressive-Integrated Moving Average Time Series Models." *Journal of the American Statistical Association* 65 (332): 1509–26.
- Ljung, Greta M, and George EP Box. 1978. "On a Measure of Lack of Fit in Time Series Models." *Biometrika* 65 (2): 297–303.